

PHP-Sensor: A Prototype Method to Discover Workflow Violation and XSS Vulnerabilities in PHP Web Applications

Shashank Gupta
Department of Computer Engineering
National Institute of Kurukshetra
Haryana, India
shashank.gupta@acm.org

B.B.Gupta
Department of Computer Engineering
National Institute of Kurukshetra
Haryana, India
gupta.brij@gmail.com

ABSTRACT

As the usage of web applications for security-sensitive facilities has enlarged, the quantity and cleverness of web-based attacks against the web applications have grown-up as well. Several annual cyber security reports revealed that modern web applications suffer from two main categories of attacks: Workflow Violation Attacks and Cross-Site Scripting (XSS) attacks. Presently, in comparison to XSS attacks, there have been actual restricted work carried out that discover workflow violation attacks, as web application logic errors are particular to the expected functionality of a specific web application.

This paper presents PHP-Sensor, a novel defensive model that discovers both the vulnerabilities of workflow violation attack and XSS attack concurrently in the real world PHP web applications. For the workflow violation attack, we extract a certain set of axioms by monitoring the sequences of HTTP request/responses and their corresponding session variables during the offline mode. The set of axioms is then utilized for evaluating the HTTP request/response in online mode. Any HTTP request/ response that bypass the corresponding axiom is recognized as a workflow violation attack in PHP web application. For the XSS attack, PHP-Sensor discovers the self-propagating features of XSS worms by monitoring the outgoing HTTP web request with the scripts that are injected in the currently HTTP response web page. We develop prototype of our proposed defensive model on the web proxy as well as on the client-side for the recognition of workflow violation and XSS attacks respectively. We evaluate the detection capability of PHP-Sensor on open source real-world PHP web applications and the simulation outcomes reveal that our defensive model is efficient and feasible at discovering workflow violation attacks, XSS attacks and experiences tolerable performance overhead.

Keywords

Axioms, HTTP, PHP, Web Application Security, Workflow Violation Attacks, XSS Attacks

1. INTRODUCTION

In the past era, web applications have turned out to be the utmost

widespread tactic for service provision over the WWW. As the web applications get extremely implanted in business related events and necessary to provide smooth performance, the plan of strategy and execution of web applications is getting extra complex. Moreover, the growing attractiveness and level of complication make web applications a main aim for attackers in the era of cyber world. In the current study [10], 63% of all Internet exploits are related to cyber-attacks on the modern web applications. Normally vulnerabilities contained by web applications are generally categorized into two main parts [6]: Injection attacks and Workflow violation attacks. The former attack results from inadequate or inaccurate sanitization of user-supplied input data, which permits numerous injection vulnerabilities (e.g. Cross-site Scripting (XSS) etc.), where poorly written malicious code is embedded into web applications. The later attack is instigated by inadequate state check verifications, which permits critical information and obstructive functions to be retrieved at incorrect states of web application.

In comparison to XSS attacks, there have been actual restricted amount of work that discover workflow violation attacks. The key challenge originates from the statistic that workflow violation attacks are particular to the expected functionality of a particular web application. It has been observed that in year 2011, a massive volume of information regarding credit card was disclosed due to a workflow violation vulnerability contained by the web application of citigroup [11]. Moreover, it was highlighted in the 2014 Symantec Internet Security Threat Report [12] that 1 in 8 web sites had critical vulnerabilities. Moreover this report also highlights that in December 12th, 2014 Cross-site scripting (XSS) vulnerability discovered in wind turbine control application.

1.1 Workflow Violation Attacks

A workflow is an explicit series of exchanges that a user has to execute to finish a transaction. Normally, a user selects a item for consumption, offers shipping and payment information, and examines the order before final approval. All interactions are controlled by distinct fragments of code and change the values of session variables. Typically, web applications verify the accuracy of an interaction series by verifying the session variables at all phases.

Workflow violation attacks (also known as State Violation Attacks) abuse the logical faults in modern web applications to violate the expected workflow of the web application. The expected workflow of a web application exemplifies a prototype model of the likely user dealings with the web application.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

FISP '15, May 18–21, 2015, Ischia, Italy.

Copyright 2010 ACM 1-58113-000-0/00/0010 ...\$15.00.

<http://dx.doi.org/10.1145/2742854.2745719>

1.2 Cross-Site Scripting (XSS) Attacks

Cross Site Scripting (XSS) is known as the leading security problem currently faced by the web application developers and the most general attack that attackers exploit to replicate the malicious code to victim's web application [9, 20].

XSS involves the insertion of a malicious code into a victim's Web application so that in future, when a victim browses the Web application, the malicious script code is executed by the browser of victim. XSS is a malicious attack vector that is escalating exponentially in prominence, because with the beginning of Web 2.0 and the growing nature of the Social Networking Web sites, more and more Web sites are permitting the web users to upload the information to web applications, repeatedly in the shape of comments or messages [21, 22].

In this paper, we present PHP-Sensor, a novel defensive model for the discovery of workflow violation attacks and XSS attacks in the real-world PHP web applications. For the detection of workflow violation in web application, PHP-Sensor operates in two phases: offline phase and recognition phase. In the former phase, the intended specifications of the web application are extracted by perceiving the HTTP request and response series and the resultant values of session variable during the period of attack-free execution. In the later phase, the deduced model is utilized to assess every incoming HTTP request and outgoing HTTP response and discover any variations. On the other hand, for the detection of XSS vulnerabilities in PHP web applications, our defensive model circumvents the dissemination of XSS worms by observing the outgoing HTTP web request that transmit self-propagating payloads. We extract all HTTP request on the client-side user interface and match them with the injected scripts.

The remainder of the paper is structured as follows. Related work is presented at Section 2. Our Proposed design is described in Section 3. Section 4 illustrates the implementation of our proposed technique. Evaluation results are highlighted in Section 5. Finally in Section 6, we have concluded our work and discuss further scope.

2. RELATED WORK

By scrutinizing the high influence of workflow violation attacks and XSS attacks on the modern web applications, a substantial measure of related work on numerous state-of-art techniques has been performed. BLOCK [1] is a black-box approach for deriving the specifications of web applications and discovering the state violation attacks. Doupe et al. [2] proposed a methodology to construct a black-box model of the internal state machine of web application by transferring HTTP request repetitively and monitoring the variation in HTTP responses. However this method incurs intolerable runtime overhead. LogicScope [3] have designed the logic of a web application with the help of a Finite State machine (FSM) and discover the logic flaws as a result from difference between the intended FSM and an actual implemented FSM. NoTamper [4] had presented a methodology for the detection of server-side HTTP parameter tampering vulnerabilities in real-world web applications through black-box investigation. RoleCast [5] is information-flow dependent technique that derives an access control invariants dependent on the values that are usually verified before executing restrictive events. Although, Swaddler [6] is a server-side technique that examines the complete description of internal state of PHP-based web applications using several anomaly models for the discovery of attacks against the workflow of web applications. Waler [7]

developed a white-box tool to evaluate the logic flaws in real-world web applications. WAPTEC [8] detects parameter tampering vulnerabilities and generates the exploits by creation to determine those vulnerabilities.

However, all these state-of-art techniques need source code of web application for instrumentation, which in turn makes them strictly coupled with the several advanced development languages of web application (e.g., PHP, CSS, JSP etc.) and other implementation specifics. Currently, there exist hardly any technique that deals with the discovery of workflow violation attacks and XSS attacks simultaneously in the real-world PHP web applications. However numerous researchers in the past have focused on these two vulnerabilities separately, that too suffers from various false positives, false negatives and intolerable performance overhead. So motivated by identified research gaps as discussed in the several state-of-art techniques, we present PHP-Sensor, a novel defensive model which discovers the vulnerabilities of workflow violation and XSS attack simultaneously in real-world PHP web applications. We developed a prototype system of our model and the evaluation results revealed that our method is capable to detect these attacks with tolerable minimum overhead.

3. PROPOSED DESIGN: PHP-SENSOR

Figure 1 highlights the abstract view of the design of PHP-Sensor.

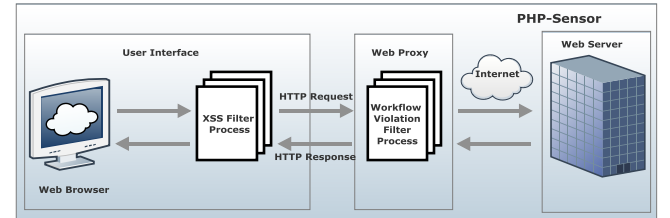


Figure 1. Abstract View of PHP-Sensor

The client-side user interface XSS filter process thwarts the dissemination of XSS worms by observing HTTP requests and responses that transmit self-propagating XSS payloads. On the other hand, the workflow violation filter process is deployed in the web proxy. This filter process discovers the workflow violation attacks by deriving the expected flow model of web application via perceiving the communications between the web browser and the server. The proposed PHP-Sensor defensive model for discovering workflow violation attacks has two main phases: offline and recognition phase. On the other hand, for the detection of XSS worms on the client-side user interface, PHP-Sensor operates only in recognition phase.

3.1 Design Model Framework

In this paper, we have only conducted the offline phase for deriving a set of axioms on web proxy and in recognition phase; we discover the workflow violation attacks in the observed HTTP request/response that violates the derived axioms. However, we have not conducted any offline/training phase for deducing the set of axioms for discovering XSS attacks on the PHP web applications. PHP-Sensor simply discovers the XSS attacks on the client-side user interface by observing HTTP requests and responses that transmit self-propagating XSS payloads. XSS filter process deployed at the client-side user interface captures all HTTP requests and responses and match them with currently injected scripts to discover XSS attacks. In nutshell, PHP-Sensor,

initially extracts the set of axioms in offline phase and subsequently utilized the same set of axioms in the recognition phase for discovering workflow violation attacks. In addition to this, PHP-sensor discovers the XSS attacks on the client-side user interface by extracting HTTP request and responses that may contain the malicious XSS payload.

For the discovery of workflow violation attacks, firstly, in the offline phase, the intended specifications related to the application are extracted via perceiving the HTTP request and response series and the resultant values of session variable. Secondly, in the recognition stage, the deduced model is utilized for assessing every HTTP request and HTTP response and discovers several variations. In the context of stateless environment of HTTP protocol, values of session variables are clearly well-defined in applications to preserve the state of a session. Normally, two techniques are utilized for retaining states of session variables: Firstly, from time to time, session states are embedded in cookies, web address and stored at the client-side web browser. Secondly, on the other hand, the web server holds the states of session and facilitates with a session identifier to the browser for indexing resultant states of session. In both cases, corresponding session states could be recovered at execution time for every HTTP request independent of the implementation of web application. We have described the actions of web application in the shape of three possible types of axioms which are as follows:

- 1) *Type 1 input axioms*: Usually input of web application comprises of the HTTP request and the corresponding variables of session whenever the HTTP web request message is initiated. This kind of axiom represents the association between the HTTP web request message and the corresponding values of variables of session.
- 2) *Type 2 input/output axioms*: This kind of axiom represents the association between the HTTP web request message and HTTP web response messages as well as the alterations in the variables of session when the HTTP web request message is managed.
- 3) *Type 3 input/output sequence axioms*: Normally, while the associated variables of session are not appropriately well-defined, a new third category of axiom is defined. This kind of axiom establishes the correlation between successive HTTP request/response sets.

3.1.1 Web Application Model

Normally, web application is considered to be a stateless system SL that takes a value of input R_{in} and generates an output value R_{out} , which is further articulated as

$$S_L(R_{in}) = R_{out}$$

The input R_{in} comprises of an HTTP web request message and a variety of name or value pair $S(R_{in})$ of session variables. In order to simplify recognition ability, we additionally separate a HTTP request into two constituents: An HTTP request key $K(R_{in})$, which consists of method of HTTP request and a variety of set of input parameter name/value $P(R_{in})$ and a requested file. Likewise, an output comprises of a HTTP response web page and a number of name/value pair $S(R_{out})$ of session variables. If we allocate a distinctive ID to every static pattern of web page or template, an HTTP response page can be represented as a HTTP response key i.e. $v(R_{out})$ and a variety of pairs of output parameter name or

value $Q(R_{in})$. In the following subsequent section, we describe the method of symbolizing a web page into a pattern of web pages or templates with a set of parameters of output.

3.1.2 PHP Web Page Representation

To represent PHP web page, we initially recover the web page patterns (P) by using all the perceived web pages (W). Now, assume a web application's page $w \in W$, we categorize it into some of likely template (L) and recover various output parameters (O) consequently. Procedure for drawing out patterns of web page or template embedded in web page has been obtainable in current state-of-techniques [13, 14]. Although, we utilize some of the techniques from TEXT [13] that highlights the DOM tree of a page reflecting the required paths. Our web page pattern mining technique comprises subsequent four stages. Stage first and second are analogous to TEXT and stage third and fourth are planned to accomplish the aim of extraction of patterns web page in the proposed framework PHP-Sensor.

1. *Conversion*: The DOM organization of a web application's page 'w' is initially converted into a variety of possible paths P_w . At this point, we put emphasis on those possible paths that reach towards the text nodes that in turn transfer the information to the web browser of the client inside the web pages.
2. *Cropping*: Crop the web page patterns from all possible paths, probable paths that point towards the dynamic content information should be cropped. For this, we describe the provision of possible path as the quantity of web pages in 'W' that cover the probable path. As the existence of possible path that fits to a web page pattern is usually advanced, paths comprising small support are utmost possible dynamic content information and must be cropped. In place of every web page 'w', the least possible support threshold T_h which is defined as the way of the existence of possible paths, which are embedded in the web page. Note that utilizing same threshold value for every web page is incorrect as every web page pattern may produce dissimilar quantity of web pages. As the probable paths comprising support lesser than the value of threshold are cropped, every web page is stated in terms of category of "CRUCIAL" paths. We utilize CRCP (w) to represent the quantity of crucial paths embedded in web page 'w'.
3. *Grouping*: Usually two web pages are possibly produced from the similar web page pattern if they consist of related set of possible crucial paths. The similarity (SIM) distance between two pages w_i and w_j is well-defined as follows:

$$SIM(w_i, w_j) = CESP(w_i, w_j) / \sqrt{CRCP(w_i) * CRCP(w_j)}$$

Where CCRCP (w_i, w_j) is the amount of common crucial paths enclosed in web pages w_i and w_j . We then execute ordered agglomerative grouping on all web pages grounded on the above similarity relationship metric. Every resultant group relates to an extracted web page pattern. The set of possible essential path of a fresh web page pattern is the intersection of route sets from the two web page patterns, which are combined together.

4. *Parameterization*: For every web page in 'W', subsequently after excluding the possible essential paths embedded in the web page pattern it fits to, the residual paths in its set of path fit to parameters of related output.

3.1.3 Axioms Extraction

We produce following three varieties of axioms.

Type 1 Axiom

The associated input parameters using the similar request key r are assembled together. We obtain following categories of rules for every HTTP web request key r .

- A collection of related variables of session $S_v(r)$, which are permanently present.
- A variety of input parameters $P_i(r)$ that are permanently present.
- For a particular associated variable of session $s \in S_v(r)$, its corresponding value is retrieved through the enumeration set $V(s, r)$.
- For particular associated parameters of input $p \in P_i(r)$, its corresponding value is accessed through the enumeration set $V(p, r)$.
- The corresponding value of an associated parameters of input $p \in P_i(r)$ is constantly equivalent to of any value of variable of sessions $s \in S_v(r)$.

Type 2 Axiom

The pairs of input or output by the similar pair of key (r, v) are assembled together. We initially obtain similar category of axioms same as type 1 for an associated pair of key. We likewise obtain two new axioms for every associated pair of key (r, v) :

- The associated value of output constraint is constantly equivalent to the input value of constraint and any associated variable of session. This axiom reveals the flow of data inside the web application.
- The session state is unaffected.

Type 3 Axiom

For every request key r , we obtain the resulting axiom: A collection of key pairs of input or output, which continuously lead the key of HTTP web request in individual session.

3.1.4 Workflow Violation Attack Detection

All HTTP web request key r is linked through a variety of axioms, containing type 1 as well as type 3 axioms. Each pair of input or output (r, v) is also linked through a class of type 2 axioms. For recognition, every axiom is converted into an estimation function that activates on a set of associated input pair. If the set of input pair follows an axiom, the estimation function yields true value. Else, the estimation function yields false value. The recognition phase of detection of workflow violation by our proposed framework PHP-Sensor is accomplished in the following two cases:

(1) *Authenticating the input R_{in}* : In this case, the HTTP request is acknowledged only if the request key has been perceived and each axiom linked with it is fulfilled. Else, the HTTP web request is obstructed.

(2) *Authenticating the pair of input or output (R_{in}, R_{out})* : In this case, the HTML page is transmitted to web user

only if the resultant pair of key set has been perceived and every axiom linked with it is fulfilled. Else, the HTML web page is jammed.

3.2 XSS Worm Discovery Method

Following are the steps which we put forward to discover the XSS malwares:

1. We capture every HTTP web request that may perhaps comprise the malware of an XSS payload. We obtain values of parameter from every captured web request. By utilizing such values of parameter, we obtain URI links that is further linked to malicious files of JavaScript.
2. If the injected URI links occurs in the HTTP request message, we transmit asynchronous requests to recover JavaScript files in response to the derived URI links. Remember that, we cannot start our recursive decoding procedure (Step 4) till we obtain all HTTP responses from remote location of servers. We designate the category of extracted parameter-related values and recovered set of JavaScript files P .
3. We capture the scripts from the Document Object Model (DOM) tree of the existing Web page. Subsequently, we obtain files of JavaScript, which are placed into the present web page, from collected HTTP web response messages. We designate the obtained set files of scripts and stored external files set D .
4. In this step, we utilize a recursive decoder on the extracted code from set P as well as set D . We recursively iterate the recursive decoding method till we discover no encoded script.
5. Lastly, we utilize a HTTP response variation detector to match code from set P with code from set D in hunt of identical code, which specifies the possible dissemination actions of an XSS malware. If we identify comparable code, we linked the malicious web request and alert the web user.

The next section illustrates the fine points of our XSS worm discovery method. It describes that how we obtain values of parameter from parameter value selector component, how we extract the scripts from numerous possible locations, recursive decoding process for scripts, detection process of variation in the scripts and finally filtering process in the final HTTP response.

3.2.1 Parameter Value Selector

The malware of an XSS payload may possibly be transmitted in the shape of text message or in the form of a URI web link directing to the outward file kept on a remote location of server. In both cases, the text message or the URI web link requires to be implanted in the parameter-related values of an outbound web request for the sake of transmission. The obtained values of parameter and recovered JavaScript files create set P .

We initially obtain values of parameter from the demanded URI. After this, we observe the request function of the outward web request. If POST function is utilized, we obtain the body of the request and then obtain extra values of parameter from it.

3.2.2 Script and File Extractor

This component will analyze the source code of Google Chrome web browser not only for extracting the script content but also for determining all possible numerous ways of embedding JavaScript into an HTML web page. Initially, the JavaScript extractor

component will search for those scripts which are executed in an automated manner on loading of the web page. Secondly, this component will discover the event handlers, which will only execute on user interaction. Lastly, this component detects the JavaScript URL link scripts, which will again be executed on user click.

Automated Executed Scripts comes under those categories of JavaScript code which executes automatically on the loading of a web page. User-Interaction Scripts comes under the category of event handlers, which only executes on user interaction. Lastly URL Embedded Scripts only executes when clicked upon a JavaScript URL link. We reflect on several general cases where XSS attack vectors might be present: Inline Scripts, Script Insertion Using Local Source Files, Script Insertion Using Remote Source Files, Event Handler Code and URL Attribute Values

3.2.3 Recursive Decoder

The recursive decoder decode every possible obtainable values of parameter, recovered external files of JavaScript, obtainable scripts of DOM, and accumulated external files of JavaScript. For automating the technique of recursive decoding, recursive decoder utilizes a regular expression for all possible schemes of encoding. It is likely that an obfuscator of JavaScript uses compound layers of encoding practices.

3.2.4 HTTP Response Variation Detector

This component is utilized to match code from set P with code from set D in hunt of identical code, which specifies the possible dissemination actions of an XSS malware. If we identify comparable code, we linked the malicious web request and alert the web user.

4. IMPLEMENTATION

We implement the prototype of our proposed recognition system PHP-Sensor, whose one part is implemented as web proxy in the form of a workflow violation filter process for detecting the workflow violation attacks and the other part of PHP-Sensor is implemented on the client-side user interface in the form of a XSS filter process for the discovery of XSS attacks. We have designed a cross-platform Google Chrome extension to discover XSS worms on the client-side. PHP-Sensor is capable enough to trace the precise session-related files, indexed by the unique session ID inside the HTTP request, and read the session-related information of user. In workflow violation detection process, PHP-Sensor can be worked in two phases: offline and recognition.

4.1 Offline Phase

The modules of the PHP-Sensor in the offline phase are presented in Figure 2. Each time a HTTP web request or a HTML web page is captured, the HTTP request message view composer constructs a portrait of the present session state and creates the resultant messages, which is transmitted to the attribute accumulator. After appropriate traces of attributes have been retrieved, the PHP-Sensor will execute offline learning. The pattern extorter process initially grabs the possible patterns of web templates from perceived HTML web pages, and then breaks down equally the input and output HTTP messages into the chosen design: a request or response key value associated with a set of key value sets for both types of session and parameter variables. The break

down traces of attributes is supplied into the axioms generator process, where all three defined categories of axioms are derived.

The traces of the attributes collected are converted into the required possible layout by Daikon engine and the resultant output is a set of axioms recovered for every possible declared record. All extracted axioms encompass the required specifications of web application.

4.2 Recognition Phase

Once the axioms are retrieved, the PHP-Sensor shifts to the recognition phase for discovering the workflow violation and XSS attacks, which is highlighted in the Figure 3.

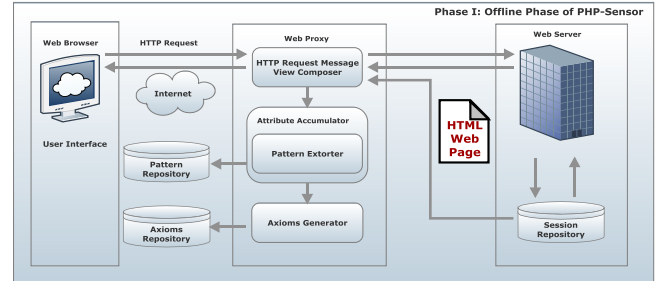


Figure 2. Offline Phase of PHP-Sensor

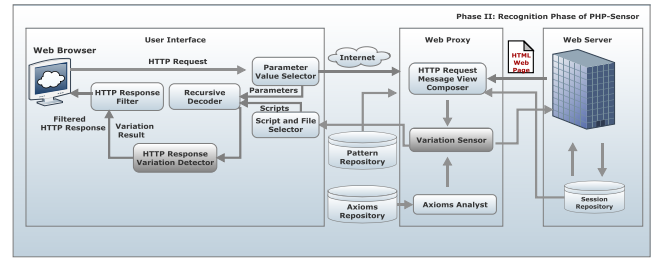


Figure 3. Recognition Phase of PHP-Sensor

The axiom analyst parses and infers the extracted axioms. In recognition phase, the HTTP request message view composer merges session-related information with the captured HTTP web request, constitutes an input message and transmits it to the variation sensor for assessment. If the input message is approved, the HTTP request message is dispatched to the web application and registered as the present input message for the web application. Else, the HTTP request message is discarded. When the HTTP request message view composer accepts a HTTP response message, if the HTTP response message is a redirection, then the consequent HTTP request will not be assessed or registered. If the HTTP response message is a HTML web page, the HTTP request message view composer allocates the HTML web page a response key created on its pattern of web template, constitutes an output and transmits it to the variation sensor, where the output is combined with the present input and assessed. If the result of the output is acknowledged, the HTML web page is transferred to the client's web browser and the key value pair is registered for the existing session of the user. Else, the HTTP web response is obstructed and the present input is canceled. In addition to this, the session of the user has ended and all registered key value pairs are washed away.

For the detection of XSS, the HTTP response from the variation sensor is supplied to the Script and File Extractor component,

which extracts all the possible scripts from the DOM tree. Now the parameter values, URI links and scripts are supplied to the recursive decoder process for decoding all the possible text until no encoded text is found. After this, the decoded text is supplied to the HTTP response variation detector to compare the extracted parameter values and URI links with the extracted scripts from DOM tree. If any suspicious similar code is detected, then the HTTP response is passed onto the HTTP response filter. The filter will redirect the HTTP request and transmit an alert message to the web browser.

5. EVALUATION SETUP AND RESULTS

We evaluate our defensive model on numerous real-world PHP web applications as listed in the Table 1 (shown at the end page of this paper). The mentioned web applications are a demonstrative illustration of different category of functionality and stages of complication that can be discovered in commonly-used PHP web applications. SimpleCms is such type of web application that permits a developer of the website to shape, write and broadcast online content for numerous users. OsCommerce is a broadly-utilized open source web application, which facilitates the attacker to straight forwardly redirect to the deposit webpage without selecting any option for the transport mode and the entire payment deposit does not consist of transport fees. WebCalender is a calendar application and is vulnerable to malicious file injection attack. PunBB is a community forum web application which provides the structure of community forums and is exposed to arbitrary file injection attack that permits the attacker to run the random malicious code. BloggIt is publically available web application of blogs, which facilitates the online users to cope a web blog, post new mails and several comments.

5.1 Recognition Effectiveness

PHP-Sensor first executes in the offline phase to conduct the training mode for obtaining the execution-related traces, produced via user simulators. The Table 2 highlights the statistics of our composed traces in the offline phase.

Then, it examines the HTTP request, web pages, retrieves HTTP request keys, patterns of web pages, key pairs and all three categories of axioms. To detect the influence of the size of training set in offline phase on the quantity of extracted rules, we fluctuate the size of the training set and compute the subsequent rules. The following figure 4 highlights the testing outcomes that we acquire for the Simplecms PHP web application. This can be clearly observed that the quantity of type I and III axioms primarily fall and at that time unite with the increased size of training set, signifying the removal of incorrect axioms observed through inadequate samples of training set. The quantity of type II axioms initially rises, since the consideration of novel state space which is not exposed using the training set of small size, and then likewise gradually converges. Keeping in this view, we utilize training set designed for every web application for which the quantity of axioms unites together.

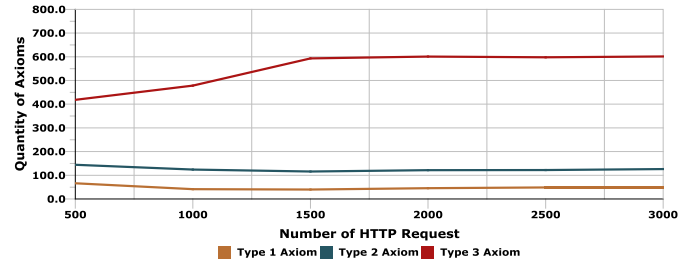


Figure 4. Experimental Result for Simplecms PHP Web Application

After this, PHP-Sensor shifts to the recognition phase. The PHP Web pages retrieved from corresponding HTTP request is produced by simulators of the user, who control the web applications. Fourteen attack examples are produced in dissimilar environments for every PHP application. The Table 3 highlights the statistics of detection results of PHP-sensor. Each exploited attack is effectively discovered by PHP-Sensor and with minimum observed false positive rate. Such statistics validates the efficiency of PHP-Sensor at discovering workflow violation attacks.

Furthermore, we examine observed false positives and discover two chief causes. The first cause is due to the partial survey of overall application executed via the simulator. The selection of state space is decided by the proficiency of simulator that PHP-Sensor can illustrate for the web application. The extra the simulator surveys, the extra precise such axioms are. This can be observed that, the error pages which are not surveyed by the simulator produce certain false positives.

In real-world scenario, the proposed PHP-Sensor could be willingly utilized and execute efficiently if certain traces exists. The additional rate of false positive is the imprecise symbolization of PHP pages. Symbolization process of web page disturbs the offline as well as recognition phase. In the offline phase, the quantity and the superiority of the deduced axioms, particularly for type 2 axiom, are strictly linked through the amount of obtained pattern of web pages. This can be observed that the quantity of type 1 and 3 rules unites very wildly, therefore, diminishes the false positive rate for HTTP request whereas type 2 axioms fetch additional false positives rate of HTTP responses. In the recognition phase, it is likely that a corresponding page is categorized into an incorrect template, which possibly obtains an unnoticed input/output pair and therefore a false positive. As obtaining the template is not our attention in this paper, we utilize some of the techniques from TEXT, which executes fine with the PHP web applications. In order to escalate the correctness and strength of symbolization of web pages, progressive algorithms or manual review could be familiarized for managing the procedure.

The recognition outcomes of PHP-Sensor too reveal the category of rules bypass by numerous attacks. Authentication violation on

Table 2. Statistics of Composd Traces in Offline Phase

PHP Web Applications	HTTP Requests	PHP Web Pages	HTTP Request Keys	Web Pages Patterns	Key Pairs	Type 1 Axiom	Type 2 Axiom	Type 3 Axiom
Simplecms	3216	3210	23	22	76	86	640	23
OsCommerce	2655	2644	18	13	39	74	128	15
WebCalender	2651	2559	22	11	45	144	456	27
PunBB	2942	2926	15	39	132	65	123	11
BlogIt	3869	3860	21	924	56	292	617	19

Table 3. Statistics of Detection Results of PHP-Sensor

PHP Web Applications	HTTP Request	Blocked HTTP Request	Blocked HTTP Response	Blocked HTTP Response	No. of Attacks Exploited	No. of Attacks Discovered	Axioms Violation
Simplecms	1709	1665	2	7	14	14	Type 1 and 3
OsCommerce	1495	1443	1	8	14	14	Type 1 and 3
WebCalender	926	917	0	0	14	14	Type 1 and 3
PunBB	1896	1878	2	1	14	14	Type 1 and 3
BlogIt	1023	1013	3	9	14	14	Type 1 and 2

inadequate examination of session variables causes the destructions of type 1 axioms which are enforced on associated session state, when HTTP requests are likely to be accepted. Furthermore they could disrupt type 3 axioms due to the omitted phase of authentication. In addition to this, input validation attacks can be discovered through type 1 axioms, if parameters of input are linked to the corresponding session variables. These attacks can too be recognized by type 2 axioms, if the resultant pages hold parameters of output which are linked through session state. Workflow violation attacks could be obstructed in similar way as authentication violation attacks, if session variables that are utilized for protecting the state transitions are not verified. If no such defending session variables are present, type 3 axioms would support to detect workflow violation attacks because of the restrictions enforced on the series of procedures.

6. CONCLUSION AND FUTURE WORK

This paper presents PHP-Sensor, a novel defensive model for discovering workflow violation attacks and XSS attacks and assesses the implementation of its prototype system on numerous open source PHP web applications. Our work discovers the workflow violation attack on the web proxy by discovering the variations in the expected and observed workflow of PHP web applications. In addition to this, we introduced a methodology which discovers the dissemination of

XSS worms on the client-side user interface by extracting all HTTP request and match them with the injected scripts. The resultant outcomes verify the efficiency of PHP-Sensor. Our defensive methodology is relevant and robust too as it is self-reliant on the source code of the web application.

We also would prefer to point out some of the drawbacks of our approach. First, PHP-Sensor cannot apply on web applications, which are development on platforms like AJAX. Secondly, PHP-Sensor has restricted ability in controlling difficult restrictions inside the database. In the future work, apart from some of the limitations our technique, we will further enhance our work to consider several other portions of internal state of PHP web application. In addition to this, we will pay attention on several methods of optimization, which will reduce the performance overhead incurred by PHP-Sensor.

7. REFERENCES

- [1] Xiaowei Li and Yuan Xue. , BLOCK: A black-box approach for detection of state violation attacks towards web applications. In ACSAC'11: Proceedings of the 27th Annual Computer Security Applications Conference, pp.247-256, 2011.
- [2] Adam Doupe, Ludovico Cavedon, Christopher Kruegel, and Giovanni Vigna., Enemy of the state: A state-aware black-box vulnerability scanner. In USENIX': Proceedings of the USENIX Security Symposium. Bellevue, WA, pp. 26-41, 2012

- [3] Xiaowei Li and Yuan Xue., LogicScope: Automatic discovery of logic vulnerabilities within web applications. In ASIACCS': Proceedings of the 8th ACM Symposium on Information, Computer and Communications Security, pp. 481-486, 2013.
- [4] Prithvi Bisht, Timothy Hinrichs, Nazari Skrupsky, Radoslaw Bobrowicz, and V. N. Venkatakrishnan., NoTamper: Automatic blackbox detection of parameter tampering opportunities in web applications. In CCS'10: Proceedings of the 17th ACM Conference on Computer and Communications Security, pp. 607-618, 2010.
- [5] Sooel Son, Kathryn S. McKinley, and Vitaly Shmatikov., RoleCast: Finding missing security checks when you do not know what checks are. In OOPSLA'11: Proceedings of the 26th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications. Pp. 1069– 1084, 2011
- [6] Marco Cova, Davide Balzarotti, Viktoria Felmetsger, and Giovanni Vigna., Swaddler: An approach for the anomaly-based detection of state violations in web applications. In RAID'07: Proceedings of the 10th International Symposium on Recent Advances in Intrusion Detection. pp. 63–86, 2007.
- [7] Viktoria Felmetsger, Ludovico Cavedon, Christopher Kruegel, and Giovanni Vigna., Toward automated detection of logic vulnerabilities in web applications. In USENIX'10: Proceedings of the 19th USENIX Security Symposium, pp. 481-486, 2010.
- [8] Prithvi Bisht, Timothy Hinrichs, Nazari Skrupsky, and V. N. Venkatakrishnan., WAPTEC: Whitebox analysis of web applications for parameter tampering exploits construction. In CCS'11: Proceedings of the 18th ACM Conference on Computer and Communications Security, pp. 575–586, 2011.
- [9] SUN, F., XU, L., AND SU, Z. Client-side detection of XSS worms by monitoring payload propagation. In ESORICS, M. Backes and P. Ning, Eds., vol. 5789 of Lecture Notes in Computer Science, Springer, pp. 539–554, 2009.
- [10] Symantec internet security threat report 2009. Retrieved from: <http://www.symantec.com/business/threatreport/>.
- [11] Citigroup credit card information leakage in 2011. Retrieved from: <http://www.wired.com/threatlevel/2011/06/citibank-hacked/>
- [12] Symantec Corporation. Symantec Global Internet Security Threat Report, vol. 19, 2014.
- [13] C. Kim and K. Shim. Text: Automatic template extraction from heterogeneous web pages. IEEE Trans. Knowl. Data Eng., Vol.23, Issue 4, pp. 612–626, 2011.
- [14] D. C. Reis, P. B. Golgher, A. S. Silva, and A. F. Laender. Automatic web news extraction using tree edit distance. In WWW '04: Proceedings of the 13th international conference on World Wide Web, pp. 502–511, 2004.
- [15] Simplecms: Simple Content Management System: Retrieved from: <http://www.couchcms.com/>
- [16] OsCommerce Inc. Retrieved from: <http://www.oscommerce.com/>.
- [17] WebCalender 1.2 Retrieved from: <http://sourceforge.net/projects/webcalendar/>
- [18] PunBB 1.4: Retrieved from: <http://punbb.informer.com>
- [19] PHP-blogit: Retrieved from: <http://sourceforge.net/projects/php-blogit/>
- [20] Shashank Gupta, Lalitsen Sharma et al. "Prevention of cross-site scripting vulnerabilities using dynamic hash generation technique on the server side". International journal of advanced computer research (IJACR), pp.49-54, 2012.
- [21] Shashank Gupta, Lalitsen Sharma "Exploitation of Cross-site Scripting (XSS) vulnerability on Real World Web Applications and its Defense" International journal of computer applications (IJCA), pp. 28-33, 2012.
- [22] Shashank Gupta, B.B. Gupta, "BDS: Browser Dependent XSS Sanitizer", Book on Cloud-Based Databases with Biometric Applications, IGI-Global's Advances in Information Security, Privacy, and Ethics (AISPE) series, pp. 174-191, USA, 2014

Table 1. Details of Real-World PHP Web Applications

PHP Web Application	Explanation	Identified Vulnerabilities	Quantity of PHP files
SimpleCms	Content Management System	Authentication Bypass Attack	22
OsCommerce	e-Commerce Application	State Violation Attack	532
WebCalender	Online Calendar and Event Management Application	File Inclusion Attack	123
PunBB	Community Forum Application	File Injection Attack	66
BloggIt	Blog Application	Authentication Violation and Injection Attack	24